

Enabling Green and Delay-Aware 5G Edge Through Shared Cloud-Native Network Functions

Ömer Zekvan Yılmaz^{ID} and Fatih Alagöz^{ID}

Abstract—Satisfying the resource requirements of Network Functions (NFs) in an environmentally friendly and cost-effective way is a challenging task for cloud providers. The placement and resource allocation procedures for these NFs should address the Quality of Service (QoS) constraints, which result in ‘underutilized resources’. Network slicing offers solutions to reduce costs through resource sharing; however, sharing NFs, which obviously contributes to achieving QoS in a cost-effective way, has not been adequately considered in the literature. Moreover, the existing research on shared NF placement so far mostly uses NP-Hard algorithms. In this paper, we propose a cloud-native NF (CNF) sharing model and a heuristic (D-CNF) that is delay-aware and cost-effective for 5G edge networks by extending our previous work, CNFSH (Yılmaz and Alagöz, 2023), which focuses on NF sharing in the 5G core without considering delay. In addition, we introduce a performance isolation mechanism that maintains the high availability (HA) of high-priority slices while exploiting shared NFs. We also propose a novel metric, the ratio of underutilized resources to allocated resources, clearly showing the effect of sharing. This ratio is around 30% with NoShare schemes, while less than 0.4% is achieved with D-CNF, indicating a green 5G implementation that serves more network slices with less resource consumption.

Index Terms—5G, edge, network slicing, sharing, placement, performance isolation, delay.

I. INTRODUCTION

THE GROWING need for applications with high throughput and low latency is driving service providers to improve NF placement methodologies in cloud settings with minimal costs. One of the main factors supporting cost-effective cloud computing is the sharing of NFs amongst network slices, which is already proposed by [2], [3], [4], [5]. The idea behind NF sharing is the largely underutilized resources to achieve QoS requirements, as shown in Fig. 1, while further exploitation of these resources is possible.

NF placement intends to deploy the NF in the optimum location such that QoS requirements are met in a cost-effective way. In order to deploy an NF with a low delay requirement, shortest-path communication links

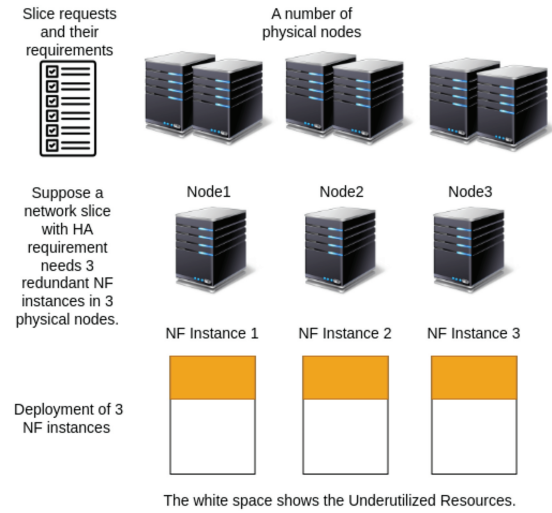


Fig. 1. Traditional NF Placement with HA.

should be established. Since the number of these shortest-path links is limited, maximizing resource utilization through sharing is an effective way to reach performance objectives.

The same applies to bandwidth requirements. Among the available resources to deploy an NF, there are locations that lack bandwidth capacity, while other locations in other computing nodes might provide higher bandwidth. Therefore, applying the appropriate NF placement policies is crucial for bandwidth requirements as well.

However, due to delay constraints, the resource consumption of NFs increases compared to delay-tolerant scenarios. The NFs in a chain require to be placed in close proximity to each other so that communication overhead is reduced. In this study, we demonstrate how delay-aware services require more resources and how NF sharing reduces these requirements and helps to save energy.

Meanwhile, network slicing can be defined as the virtualization and sharing of hardware and software resources among users. This yields virtual networks, virtual storage, virtual CPUs, and virtualized NFs as well. In this paper, we further virtualize NFs by sharing them among slices in order to increase resource utilization and achieve QoS goals with lower costs.

5G networks leverage Network Function Virtualization (NFV) and Software Defined Networking (SDN) to gain flexibility and adopt a service-oriented approach. The use of these paradigms allows network slicing to satisfy a broad

Manuscript received 15 January 2024; revised 7 June 2024 and 27 August 2024; accepted 29 August 2024. Date of publication 3 September 2024; date of current version 21 May 2025. The editor coordinating the review of this article was F. Jalali. (Corresponding author: Ömer Zekvan Yılmaz.)

Ömer Zekvan Yılmaz is with the Wireless Communication Systems Directorate, ULAK Communications Inc., 34906 İstanbul, Türkiye, and also with the Department of Computer Engineering, Boğaziçi University, 34342 İstanbul, Türkiye (e-mail: omer.yilmaz@ulakhaberlesme.com.tr).

Fatih Alagöz is with the Department of Computer Engineering, Boğaziçi University, 34342 İstanbul, Türkiye.

Digital Object Identifier 10.1109/TGCN.2024.3454190

range of use cases, like high reliability for autonomous vehicles or ultra-low delay for remote surgeries. This requires highly scalable and reliable NFs. Detailed 5G architecture and specific NF requirements are discussed in Section III and in [6], [7], [8], [9].

Deploying 5G NFs in cloud-native environments is a cost-effective solution for mobile network operators. As mentioned by Arouk and Nikaein [10], containerization is one of the design principles of a cloud-native architecture. Frangoudis et al. [11] show that containers achieve lower response times due to having less interaction with the operating system and network I/O operations compared to virtual machines (VMs). Moreover, in [11], it is shown that the start time of a container is around 1 second, while it takes around 11 seconds for a VM, an indication of the latency required for the scaling and failover scenarios in both cases. These evidences clearly show that CNFs surpass VNFs in satisfying 5G use cases.

One of the concerns about sharing of NFs is the security and privacy implications. Obviously, isolation among network slices and prevention against denial of service attacks, side channel attacks, etc. are necessary in the case of NF sharing [12], [13]. Taking into account these security precautions, the sharing of 3G/4G/5G NFs has been devised in 3GPP releases [14] and other studies [6], [15]. Based on our previous work [16], [17], the analysis of attack vectors and defense mechanisms for NF sharing is a future work.

Moreover, performance isolation, which implies both the security precautions and the performance guarantees of individual slices, is another principle of network slicing. This aspect is mostly ignored or even sacrificed in previous studies. Zharabad et al. [6] use shared VNFs and shared CNFs for cost-effective 5G core networks. However, Zharabad et al. [6] propose to trade off isolation and sharing if necessary. In this paper, we show that using a load balancing mechanism and orchestration capabilities in cloud-native environments, performance isolation is achieved with negligible costs.

In this paper, we further develop our previous study, CNFSH [1], which uses a priority-aware LB for placement and sharing of CNFs in 5G core networks. We extend CNFSH [1], with a delay-aware D-CNFSH model, for the placement of shared NFs in a 5G edge setting. In addition, this paper considers the availability, performance isolation, and delay requirements of network slices at the same time. The advantage of shared NFs in satisfying these requirements together has not yet been adequately considered in the literature. Therefore, our contributions in this paper are the following:

- 1) An availability- and delay-aware model and a heuristic (D-CNFSH) are proposed to solve the shared NF placement problem.
- 2) To our knowledge, D-CNFSH implements the first performance isolation mechanism for NF sharing.
- 3) We evaluate the performance of D-CNFSH using 10 core and 2 Radio Access Network (RAN) NFs. To our knowledge, D-CNFSH is the first NF sharing mechanism that targets 12 realistic 5G NFs, compared to previous studies such as [2], [6].

- 4) A new metric, the ratio of underutilized resources to allocated resources, is proposed.
- 5) Improvements in the CNFSH code base [1], including the use of in-memory allocations, resulting in faster processing and lower energy consumption.

II. RELATED WORK

In this section, previous studies related to NF placement and network slicing are evaluated.

Dietrich et al. [18] improve load balancing and acceptance rates by using a linear programming approach for NF placement. Meanwhile, CloudScope was built on top of the Xen hypervisor by Chen et al. [19], who used an online Markov Chain model for online prediction of the performance of co-resident VMs. In order to lower deployment costs and improve QoS, Bushnaq et al. [20] propose an automatic network slicing optimization framework.

With the adoption of containerization, research on CNF placement has increased. Taking availability needs into account, Arora and Ksentini [21] propose a dynamic resource allocation approach. It appears that deploying redundant copies of the NFs is necessary to ensure availability as a result of placing the NF within QoS limitations. The concept of ‘underutilized resources’ arises from this correlation, suggesting the potential for further performance improvements.

In order to evaluate this opportunity, virtual NF (VNF) sharing is used by Mohamad and Hassanein [2], resulting in 47% more satisfied requests than the model without sharing. The resource usages of the network slices are considered static in [2], and their dynamic nature is not taken into account. Certainly, it requires further resource management to make the transition to a dynamic environment, which we are aiming for in this paper. Additionally, [2]’s solution is NP-hard, which will result in more delays in large-scale slice scheduling.

Truong-Huu et al. [22] have developed a more refined sharing-aware strategy that takes into account the traffic capacity and resource requirements of both shared and nonshared VNFs, based on the functions of each VNF. Meanwhile, Vittal and Franklin [23] use load balancing in kernel space to ensure the HA of network slices in 5G, but without NF sharing. In [23], performance was measured by the ratio of preventing the occlusion of User Service Requests (USR), achieving 60%, and the average response time of these USRs, which is reduced by 50%.

III. BACKGROUND

A. CNF Orchestration

The concept of ‘cloud-native’ refers to an architecture created especially for cloud settings. CNFs employ technologies such as microservices, containerization, and orchestration to realize such an architecture [10], allowing dynamic deployment and scaling. An orchestration platform called Kubernetes [24], [25] ensures dependable and accessible CNF execution. The QoS criteria for CNFs include HA, low latency, and high bandwidth. While these specifications have to deal with issues that include jitter, network congestion, and all kinds of failures. This paper focuses on an appropriate load

balancing and scaling architecture for CNFs to address these problems.

We give the following definitions in order to clarify some of the terms used in this paper. A group of containers in Kubernetes is referred to as a pod; which includes a container that serves as an atomic microservice, and other containers that handle secondary tasks like managing and maintaining the main container. A service object in Kubernetes serves as a gateway to a number of identical pods. The service object has four different types: ClusterIP, NodePort, Load Balancer, and External Name [26], [27]. In this paper, the pods and load balancers are among the key components of our proposed mechanism.

B. 5G Core Networks (5GCN)

The 5GCN handles a number of functions, including billing, slicing, authentication, and authorization. 5GCN components (NFs) are designed to work in cloud environments, considering paradigms such as network slicing, SDN, NFV, and microservices architecture.

The primary innovative feature of 5G technology is its ability to serve many vertical sectors using the same hardware resources. Three use cases: enhanced Mobile Broadband, Ultra-Reliable Low-Latency Communications (URLLC), and massive Machine-Type Communications (mMTC) were used by ITU-R [28] to illustrate this phenomenon. Each use case has unique requirements for throughput, delay, number of connected devices, and reliability. Different QoS are enabled on the same infrastructure using network slicing, which enables diverse requirements at a low cost. In this paper, our objective is to further optimize resource allocation to network slices and their placement on the most appropriate hardware, either by sharing an existing CNF or building a new one.

C. Network Slicing Use Case

In order to clarify network slicing and NF placement concepts, we give the example of smart factories. Smart factories rely on a multitude of applications with varying performance demands. Real-time machine vision inspection necessitates high throughput and low latency for flawless video transmission, while remote robot control requires ultra-low latency for precise and near-instantaneous movement. Traditional networks cannot satisfy these requirements simultaneously.

Network slicing is a game-changing technology that allows virtual network slices to meet a variety of application requirements. To achieve the required performance criteria, each slice can be adjusted separately. Network slicing can be used in the smart factory scenario to generate various slices for different purposes.

CNFs are crucial in this transformation. Compared to hardware-based systems or even VMs, CNFs offer greater scalability and flexibility. The lifecycle of these CNFs, including deployment, scaling, and healing, is managed via CNF orchestration. The CNF orchestrator in the example of the smart factory would assign the right CNFs to each network

TABLE I
SUMMARY OF NOTATIONS

$Av(x)$	Availability function for a given object x .
A_f^{si}	Whether $f \in F$ is actively used by $s \in S$ as a host or guest for the index i of its requested NFs list.
$B(L_{mn})$	Bandwidth capacity of the link L_{mn}
$C(n)$	CPU capacity of node $n \in N$
$D(L_{mn})$	Propagation delay of the link L_{mn}
F	Set of NF deployments
f_n	The pod of NF deployment $f \in F$ allocated to node $n \in N$
L_{mn}	The link between two nodes; $m, n \in N$
N	Set of compute nodes available for pod placement
$P(x)$	Failure probability of x
R_s	Required redundancy for NFs of slice s to satisfy RAv_s
RAv_s	Service availability required by the network slice $s \in S$
RC_f	Requested number of CPUs by the NF f
RD_s	The maximum delay allowed by the network slice $s \in S$
RF_s	Set of NFs requested by the network slice s
RT_s	Traffic flow requirement of the network slice the $s \in S$
S	Set of network slice requests
x_n^f	Whether the NF f is deployed on compute node $n \in N$
y_n	Whether the compute node $n \in N$ is up and running.
z_s	Whether $s \in S$ is a satisfied slice request.

slice. To effectively handle massive video streams, for example, high-throughput User Plane Functions (UPFs) would be implemented for the machine vision slices. To reduce signal delays, low-latency NF instances would be placed closer to robots in robot control slices. In order to maintain service continuity, the orchestrator would also automatically recover from failures and dynamically scale CNFs in response to traffic needs.

Smart factories can provide a dynamic and flexible network infrastructure that can handle a wide range of applications with different performance needs by using network slicing and CNF orchestration. This strategy opens the door to a more reliable and efficient future for smart manufacturing by ensuring service continuity through automated CNF management in addition to optimizing resource consumption.

IV. PROBLEM DEFINITION

In this section, we present the availability model, network function sharing model, and objective function as in [1] and extend our problem definition with a bandwidth sharing model and a delay model. As in our case, the NF placement problem generally aims to maximize the number of satisfied slice requests (SSR) with a fixed amount of resources, taking into account the requirements of 5G.

Let S be the set of network slice requests and RF_s be the set of NFs requested by each $s \in S$. RAv_s is the availability requirement of s . RT_s is the traffic flow requirement of s . The set of NF deployments F , includes all deployed and running NFs. Each $f \in F$ has a resource requirement RC_f . All deployments are made on a set of compute nodes N and each $n \in N$ has $C(n)$ amount of resources. The complete notation and the list of acronyms are shown in Tables I and II, respectively.

A. Availability Constraint

The availability constraint for NFs defines the acceptable downtime of an NF that can be tolerated before significantly

TABLE II
LIST OF ACRONYMS

CNF	Cloud-Native Network Function
CU	Central Unit
D-CNFSH	Delay-Aware CNF Sharing
DU	Distributed Unit
HA	High Availability
ITU-R	Radiocommunication Sector of International Telecommunication Union
LB	Load Balancer
MHSH	Availability-aware version of [2] with replicated NFs
mMTC	massive Machine-Type Communications
NF	Network Function
NGMN	Next Generation Mobile Networks Alliance
NFV	Network Function Virtualization
RAN	Radio Access Network
RC	The heuristic sorting compute nodes in terms of remaining capacity
RR	The heuristic sorting slice requests in terms of requested resources
QoS	Quality of Service
SDN	Software Defined Networking
SLA	Service Level Agreement
SSR	Satisfied Slice Requests
UPF	User Plane Function
URLLC	Ultra-Reliable Low-Latency Communications
USR	User Service Request
VM	Virtual Machine
VNF	Virtual Network Function
3GPP	3rd Generation Partnership Project
5GCN	5G Core Networks

impacting the functionality of the network slice. This constraint is crucial to ensure high reliability aspects of 5G use cases. Obviously, in order to cope with service level agreements (SLAs), network operators should prioritize resources, implement redundancy measures, and define recovery procedures.

Each computing unit, whether it is a physical machine or a VM or a container, has a probability to fail for some time. $P(f)$ resembles the failure probability of an NF f . $P(n)$ resembles the failure probability of a node n . We assume that the availability of an NF instance (pod) relies on the success probabilities of the pod itself and of the physical node that hosts the pod. Therefore, the availability of a pod residing on node n is given by Eq. (1);

$$Av(f_n) = (1 - P(n)) * (1 - P(f)^{x_n^f}), \forall f \in F, \forall n \in N, \quad (1)$$

where f_n is the NF pod residing on the compute node n . $Av(f_n)$ is the availability of f_n . x_n^f is a binary decision about whether to deploy f_n on compute node n or not.

If the availability of f_n is not sufficient for the availability constraint of f , then f should be instantiated with sufficient number of pods to comply with this constraint. Hence, the product of failure probabilities of NF pods gives the failure probability of the NF as indicated by Eq. (2);

$$Av(f) = 1 - \prod_{n \in N} (1 - Av(f_n)), \forall f \in F, \quad (2)$$

where $Av(f)$ is the availability of f .

All NFs required by a network slice should be up and running in order to consider the network slice available, which means that the product of the success probabilities of the

required NFs gives the success probability of the network slice. Hence, the availability of a network slice s , $Av(s)$, is given by Eq. (3);

$$Av(s) = \prod_i^{|RF_s|} \sum_{f \in F} A_f^{s_i} Av(f), \forall s \in S, \quad (3)$$

where $A_f^{s_i}$ is a binary decision about whether the deployed NF f is actively used by s for the index i of RF_s .

Each network slice request comes with a set of requirements, one of which is the availability constraint. The availability constraint tells us that the actual availability of a network slice, $Av(s)$, should not be less than the required availability, RAv_s , as in Eq. (4). If this is not possible, the network slice is not satisfied.

$$RAv_s \leq Av(s), \forall s \in S \quad (4)$$

B. NF Sharing Constraint

In CNFSH [1] and D-CNFSH, despite the exploitation of idle resources, the HA of the high-priority network slice (the host) is protected due to priority-aware load balancing. Ghosh and Banerjee [29] propose a priority-aware LB that suspends low-priority traffic in cases of congestion. Likewise, if one of the pods fails, our priority-aware LB routes the flow of the host towards other pods, while the low-priority guest slices are directed to a waiting queue. Guest slices are chosen such that they tolerate delay and some traffic loss until a new pod is deployed. Frangoudis et al. [11] have measured the deployment time of CNFs as 1.129 s on average on a 16-core Intel Xeon processor and 32 GB of RAM, running Ubuntu 14.10. Using this criterion and our measurements in Section V, we classify high- and low-priority network slices.

Eq. (5) shows that only a limited number of network slices are able to use the NF f with the already deployed pods.

$$\sum_{s \in S} \sum_i^{|RF_s|} A_f^{s_i} \leq \sum_{n \in N} x_n^f, \forall f \in F, \quad (5)$$

Eq. (5) shows that the number of network slices using the NF, can not exceed the number of deployed replications for that NF. Deploying one NF instance for each NF requirement is actually similar to the classical NF placement methodologies. However, our proposal with a priority-aware load balancing approach favors the network slice with high-priority to maintain its HA requirement. Thus, securing sufficient number of NF pods in case of congestion or node failures.

C. Bandwidth Sharing Constraint

The bandwidth constraint of a directed link between two compute nodes should aggregate all directed flows between the pods residing at these compute nodes. Like most of the studies in the literature [2], [22], [30], we assume that the amount of flow required by a network slice is identical between all

the NFs involved. Hence, Eq. (6) determines whether the flow between consecutive NFs can be handled by the physical link.

$$\sum_{s \in S} \sum_{i=0}^{|RF_s|-1} \sum_{f \in F} \sum_{g \in F} RT_s x_m^f A_f^{si} x_n^g A_g^{s(i+1)} / R_s \leq B(L_{mn}), \forall m, n \in N, \quad (6)$$

where R_s is the required redundancy for NFs of slice s to satisfy its availability constraint in Eq. (4). $B(L_{mn})$ is the link bandwidth capacity between nodes m and n . Since there is no direct bonding between RF_s and the deployed NFs, we need to traverse all deployed NFs twice to track the NF deployment tuples that are serving the network slice itself. The traffic flow required for an NF is divided by the number of redundant pods of that NF, so that the load is balanced and the underutilized bandwidth resources can be shared with other network slices.

D. Delay Constraint

The delay constraint of a network slice requires that the total delay of the flow that traverses all NFs of the network slice should not exceed a certain threshold. Eq. (7) determines whether the total delay is below the limit that the slice can tolerate.

$$\sum_{i=0}^{|RF_s|-1} \sum_{f \in F} \sum_{g \in F} \max(D(L_{mn}) x_m^f A_f^{si} x_n^g A_g^{s(i+1)} | m, n \in N) \leq RD_s, \forall s \in S, \quad (7)$$

where $D(L_{mn})$ is the delay of the link between nodes m and n . RD_s is the delay budget of slice s . Here we calculate the maximum total delay that a slice can experience using the replicated pods of its NFs. We assume that $D(L_{mn})$ is constant and is not affected by the bandwidth usage of the same link. Therefore, sharing of NFs, load balancing, or any traffic congestion does not affect the delay constraint in our assumption. We also assume that the delay of incoming and outgoing traffic to and from the edge network will not affect the placement decisions in our scenarios.

E. Computing Capacity Constraint

A physical node n has a limited CPU capacity. Therefore, Eq. (8) limits the total reservation of CPUs at node n ;

$$\sum_{f \in F} x_n^f RC_f \leq C(n) y_n, \forall n \in N, \quad (8)$$

where y_n is a binary decision for running the compute node n .

F. Slice Satisfaction Constraint

In this paper, we need to count the number of satisfied network slices that have been deployed with all of their NFs. A network slice is satisfied only if its required NFs are deployed and up and running. Hence, Eq. (9) determines the state of the network slice s , which indicates that the slice is satisfied if z_s is 1, otherwise it is 0:

$$z_s = \prod_{i=0}^{|RF_s|-1} \sum_{f \in F} A_f^{si} \quad (9)$$

Eq. (9), checks all deployed NFs to see if the deployed NF f is allowed to be used by s for the index i of RF_s . The product of the outcomes for all the indices in Eq. (9), enforces to have an active and deployed NF for each index i of RF_s to consider a satisfied network slice.

G. Objective Function

The objective function (Eq. (10)) is in favor of minimal resource consumption and the maximum number of SSR:

$$\min \frac{\sum_{n \in N} y_n}{1 + \sum_{s \in S} z_s}, \quad (10)$$

where the addition of 1 to the denominator is to avoid the division by zero problem.

In Section VI, we try to achieve the maximum number of SSR for a fixed number of physical nodes for each experiment. This indicates that each SSR has the same value for the objective function. As we see in Section V, we propose a special heuristic (RR) in order to benefit from the value equality of the SSRs.

V. THE PROPOSED HEURISTICS AND ALGORITHMS

The above-mentioned problem is an example of the Multiple Multidimensional Knapsack Problem (MMdKP) [31] and can be solved with Integer Linear Programming with NP-Hard complexity [32], but it is not always feasible with a number of slice requests above 200 as experimented by Vimont et al. [33]. Panigrahy et al. [34], Ang et al. [35], and many others propose heuristics using various sorting techniques to tackle bin-packing and knapsack problems. In [1], we have proposed the CNFSH algorithm, where the $O(|S| \log |S|)$ time sorting function and the $O(|S|^2)$ time search for a suitable host NF make it feasible with polynomial-time complexity. In order to further investigate the behavior of CNFSH [1], we have previously adopted two heuristics, RR and RC. Additionally, in this paper, we propose a heuristic for delay-aware NF placement and sharing (D-CNFSH).

Below, we explain our proposals, the three heuristics, and the priority-aware LB that contribute to the performance of NF placement and sharing.

A. Sorting Network Slice Requests by Ascending Requested Resources (RR)

The objective function in Eq. (10) assumes that all slice requests have the same value. Therefore, it is not worth placing a network slice with higher resource requirements instead of another slice with low resource requirements. It will consume more energy for the same profit. In this case, sorting slice requests in terms of the requested resources in increasing order is a viable solution for this scenario.

This scenario is applicable to the NF placement problem in 5G networks, as there are cases where any satisfied network slice yields the same profit for the network operator. In this case, RR will be a suitable heuristic to solve the NF placement problem.

B. Sorting Computing Nodes by Ascending Remaining Capacities (RC)

One of the criteria in knapsack problems is to leave as little waste space as possible in the computing node after placing the NF [31]. The waste space is the residual resource of the computing node, such that it is too small to place a new NF. In this respect, the heuristic RC sorts the computing nodes in terms of remaining capacities in increasing order for each network slice request iteration.

C. Sorting Computing Nodes by Descending Remaining Capacities for Lower Delay (D)

In addition to our previous work, CNFSH [1], we propose a heuristic that helps delay-aware network slices to be deployed in a 5G edge setting, which we call D-CNFSH. Most of the previous studies, like [2], use NP-Hard algorithms for delay awareness. In this paper, we propose a heuristic for delay-aware NF placement. For this purpose, the remaining capacities of the compute nodes are sorted in descending order for each network slice to be deployed, as shown in Algorithm 1. This is to support the deployment of the necessary NFs as close as possible so that the communication links between NFs result in a lower delay. In fact, the D-CNFSH-RC and D-CNFSH-RCRR (D-RC) schemes lose their advantage of sorting the nodes in the order of ascending remaining capacities when this heuristic is applied. This is because RC and delay-aware sorting heuristics offer a trade-off between the number of SSR and delay, which can be used as a design parameter for a wide range of use cases, as explained below.

In Algorithm 1, we start with sorting the slice requests in terms of the requested resources in ascending order, if it is an RR scheme. Then, we start processing the slice requests one by one. For each slice request, there will be a loop until we find that there is no space available. For an RC scheme, we sort the compute nodes in terms of the remaining capacities in ascending order. However, for schemes that are both delay-aware and RC, we follow a strategy that first looks for available resources as an RC scheme without delay awareness. If no resources are found, possibly due to a delay constraint, we sort the compute nodes by remaining capacities in descending order. This sorting is used to keep the NFs of the delay-constrained network slice in close proximity to achieve a low delay. A scheme that is delay-aware and not RC will directly sort the compute nodes in a delay-aware manner. As a result, only the schemes that are delay-aware and RC simultaneously loop twice; others loop only once.

Later, we loop for each NF f required by the network slice. Except for NoShare schemes, if the requested network slice has a low-priority, we check all previously deployed NFs of the same type with f to see if we can find some underutilized resources that are enough for f . Then, if the established NF chain so far does not violate the delay constraint of the requested network slice (Eq. (7)), the already deployed NF is

Algorithm 1 Delay-Aware Slice Scheduler

```

if Scheme == RR then
  SortByRequestedResources( $S$ )
end if
RepeatForLowDelay  $\leftarrow$  false
while  $s$  in  $S$  do
  while TRUE do
    if Scheme  $\neq$  RC or RepeatForLowDelay then
      SortByDescendingRemainingCapacities( $N$ )
    else
      SortByRemainingCapacities( $N$ )
    end if
    RepeatForLowDelay  $\leftarrow$  false
    while  $r$  in  $RF_s$  do
      Found  $\leftarrow$  false
      if Scheme  $\neq$  NoShare and Priority( $s$ ) == Low then
        while  $f$  in  $F$  do
          if Type( $r$ ) == Type( $f$ ) and
            UnderutilizedCapacity( $f$ )  $\geq$   $RC_r$  and
            CalculateDelaySoFar( $s$ )  $<$   $RD_s$  then
            Found  $\leftarrow$  true
            break
          end if
        end while
      end if
      if Found == False then
         $F.append(r)$ 
        Deploy( $r$ , Requirements( $r$ ),  $s$ )
        if DeploymentFailedDueToDelayConstraint then
           $F.pop()$ 
          RepeatForLowDelay  $\leftarrow$  true
        end if
      end if
    end while
  end while

```

shared and f is marked as satisfied. If no underutilized resource is found for f , then a new NF should be deployed.

The NF deployment algorithm shown in Algorithm 2, takes the NF f and its requirements, such as availability, CPU, memory, and delay, and the requested network slice s , as parameters, and looks for an available compute node to host the f . The algorithm first calculates the number of required replications of f that are enough to satisfy the availability requirement. Then each compute node in the list is checked to see if it can provide the required resources. Later, assuming that f is placed at that node, the delay of the previously deployed NFs of the network slice s is calculated and compared with its delay constraint. In case these checks are successful, an NF pod is deployed in this compute node. The loop is executed until all necessary replications of f are successfully deployed.

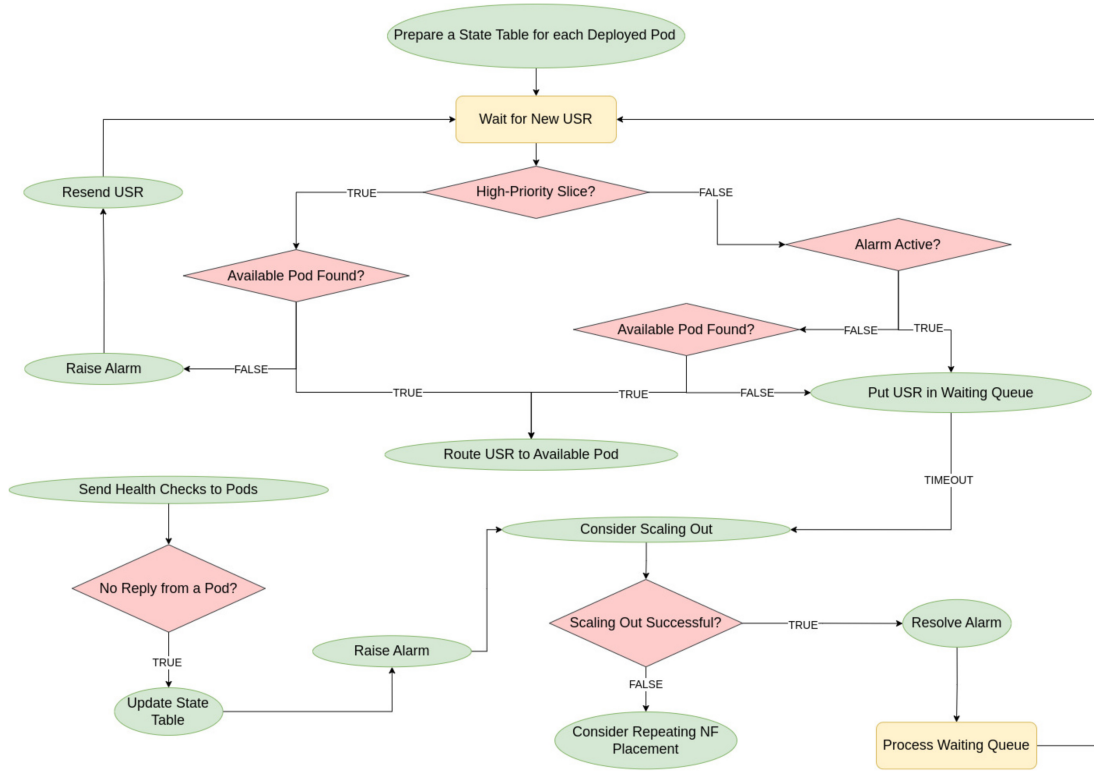


Fig. 2. State diagram for priority-aware LB.

Algorithm 2 Deploy(NetworkFunction f , Requirements(f), Slice s)

```

 $p \leftarrow \text{CalculateRequiredReplications}(f)$ 
while  $n$  in  $N$  do
  if  $\text{RemainingCapacity}(n) \geq \text{RequiredCapacity}(f)$  and  $\text{CalculateDelaySoFar}(s) \leq RD_s$  then
    Place( $f$ )
    UpdateRemainingCapacity( $n$ );
     $p \leftarrow p - 1$ ;
    if  $p \leq 0$  then
      break
    end if
  end if
end while
ReturnDeploymentResults

```

D. Priority-Aware LB

In order to protect the availability requirements of high-priority slices that can be affected by node or pod failures, and traffic congestion, as well as to maintain the performance isolation of network slices, we employ a priority-aware LB by extending [1], [29]. The complete logic and state diagram of the LB is shown in Fig. 2. First, a state table that holds the current bandwidth usage is prepared for each deployed pod. Then, the LB starts to wait for USRs from clients. Each USR is checked to see whether it belongs to a high- or low-priority slice. If it belongs to a low-priority slice, then an additional check is performed to determine whether the requested NF

is in an alarm state, or not. If the alarm is active or there is no available pod, the request is sent to the waiting queue. In this way, the LB detects any congestion, suspends low-priority slices, and raises an alarm. This takes 0.105 milliseconds (ms) in the worst case according to the measurements of Ghosh and Banerjee [29]. Otherwise, the low-priority slice is routed towards an available pod. On the other hand, if no available pod is found for the USR belonging to a high-priority slice, an alarm is raised, and the request is sent for reprocessing.

In a separate thread, the LB sends health checks to deployed pods. If a pod does not reply, the pod is marked as not available in the state table, and an alarm is raised. Alarms are also processed in a separate thread, such that SLAs are checked to see whether further resources can be allocated to the corresponding network slice. If this is the case, scaling can be triggered, and the alarm is resolved in case of successful scaling. If the scaling is not included in SLAs of the network slice, the corresponding USR can be dropped due to bandwidth overload. If the alarm is resolved the USRs in the waiting queue are sent for reprocessing, otherwise dropped out after a timeout period.

It is worth mentioning that we deploy redundant pods in order to satisfy the availability constraints of the network slices, and we make use of the underutilized resources that result from this redundancy. In Fig. 3, we demonstrate some cases that could be encountered in the lifetime of these pods. In the first column of Fig. 3, we see that one network slice is using a couple of pods. The white space resembles the underutilized resources that are already allocated to these pods. The second column shows two new network slices that are making use of these resources.

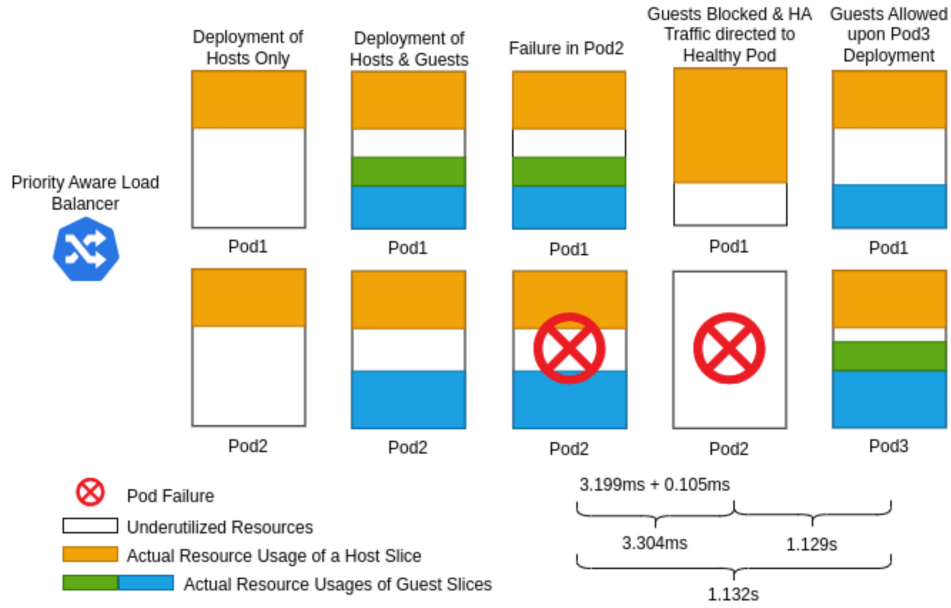


Fig. 3. Example allocation of underutilized resources to guest slices and a pod failure scenario.

Our measurements show that the round trip delay between LB and a Kubernetes pod is 1.199 ms in the worst case and 0.639 ms in the best case. If the LB sends health checks to deployed pods every 2 ms (a configurable parameter) in separate threads, a pod failure can be detected in 3.199 at worst case. Then an alarm is raised and none of the low-priority USRs are allowed to reach the deployed NF pods, as shown in the third column in Fig. 3. Later, a new pod (Pod3) is triggered and instantiated in around 1.129 seconds [11], and corresponding USRs are directed to Pod3 in around 0.105 ms for each USR in the worst case [29], as shown in the fifth column (Fig. 3). This also shows that the availability and performance isolation of these low-priority slices are not violated for more than around 1.132 seconds with D-CNFSH.

Noting that SLAs and the measurements of Frangoudis et al. [11] are used to determine the priorities of the network slices. Furthermore, if any congestion occurs due to a violation of the SLA limits, then predefined policies should be implemented, which can be detected using the LB state tables.

VI. PERFORMANCE EVALUATION

In this section, we first evaluate the performance of proposed heuristics and then the priority-aware LB.

A. Performance Evaluation of the Proposed Heuristics

For the performance evaluation of our proposed heuristics, we have extended our open source slice scheduler on GitHub [36]. For comparison, we use six non-delay-aware schemes: two NoShare models, the dynamic version of the sharing model of Mohamad and Hassanein [2] as MSHS and three CNFSH algorithms [1]; and six delay-aware schemes: D-NoShare, D-CNFSH, D-CNFSH-RC, D-NoShare-RR, D-CNFSH-RR, and D-CNFSH-RCRR.

We use MSHS [2] for comparison because it aims to exploit all the underutilized resources at a given time t , yet in a static manner. In order to make the comparison possible in a dynamic setting, we adopt our priority mechanism into MSHS.

NoShare models do not use any NF sharing logic; NoShare-RR only sorts the slice requests in terms of ascending requested resource size. The remaining algorithms first sort the slice requests by priority. No more sorting is performed in MSHS. CNFSH-RC and CNFSH-RR apply the heuristics explained in Section V. CNFSH-RCRR, applies both of them. Delay-aware schemes follow Algorithm 1. The algorithms using RR are evaluated separately for fairness.

We use four metrics to compare all these schemes: the number of SSR, the average CPU allocation per network slice, the computation time, and our proposed metric, the ratio of underutilized resources to all allocated resources. With the assumption that the memory requirements are in parallel with the computing requirements, we just use the CPU requirements in our calculations. The experiments were run on Ubuntu 20.04.6 running on an Intel Core i7-8550U CPU @1.80GHz and 16 GB of memory. An NF catalog, containing 10 sharable NF types as in [4] and the addition of 2 RAN functions, CU and DU, which are described by 3GPP [37], was used. We have also improved our open source implementation [36] with faster execution times compared to [1], as seen in Fig. 10.

In the experiments, the slice requests consist of 3-10 different NF types selected randomly from the NF catalog. User Plane Function (UPF), which has a heavy workload as devised by the study in [38], requires 8 CPUs. Other NFs require 2 CPUs. High-priority slices, within the scope of URLLC, require 99.999% availability, as in NGMN [39]. Low-priority slices, within the scope of mMTC, require less than 98% availability. All NFs and physical nodes have 0.999 availability, as experimented by Arora and Ksentini [21].

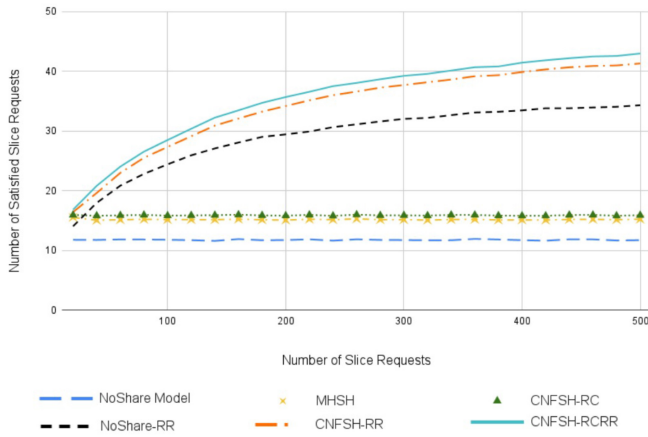


Fig. 4. Number of SSR without delay-aware heuristic vs. number of slice requests.

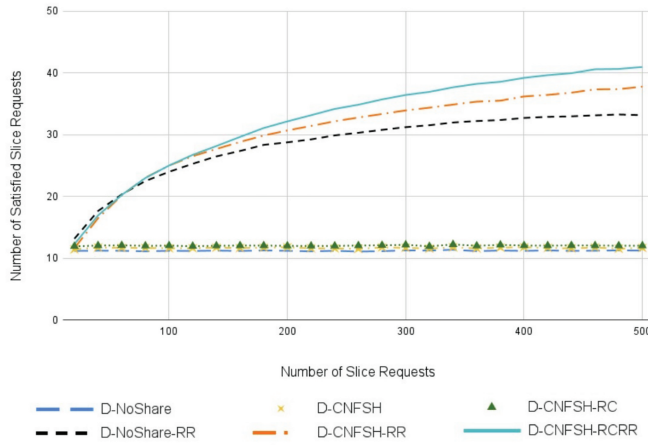


Fig. 5. Number of SSR with delay-aware heuristic vs. number of slice requests.

Various numbers of randomly generated slice requests are experimented with between 20 and 500 with step size 20. Each set of slice requests is tested for all twelve models. Each experiment is repeated 600 times, and the average values are used. Six physical nodes with decreasing capacities from 100 to 80 CPUs were simulated.

For delay-sensitive experiments, all network slices were assigned a random delay budget between 10 and 20 ms. The link delay between NFs in the same computing node is assumed to be 0, and between NFs in different computing nodes it is assumed to be 10 ms.

In Fig. 4 and 5 the y-axes show the absolute number of SSR. The x-axes in all figures show the number of slice requests. MSHH and CNFSH-RC reach around 30% and 35% higher SSR than the NoShare Model, respectively. At the same time, CNFSH-RC reaches 5% higher SSR than the MSHH model. CNFSH-RR and CNFSH-RCRR achieve 24% and 29% higher SSR compared to NoShare-RR (Fig. 4).

In Fig. 5, D-CNFSH-RC achieves around 2% higher SSR than D-CNFSH when the sorting heuristic for delay awareness is applied. The advantage of D-CNFSH-RC is due to the proposed heuristic, which tries to find a balance between

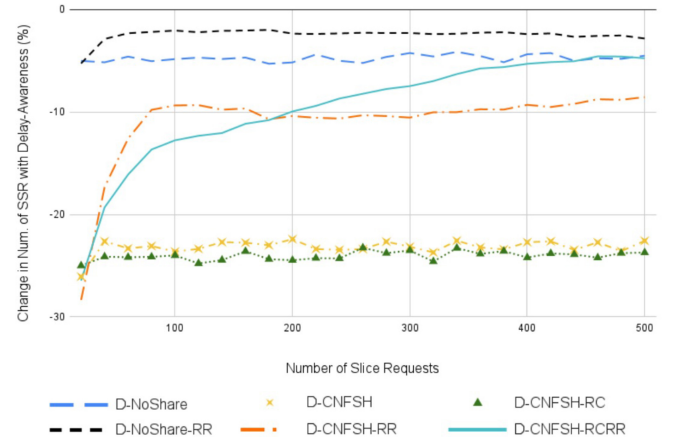


Fig. 6. Percentage of change in the number of SSR when the delay-aware heuristic is applied vs. number of slice requests.

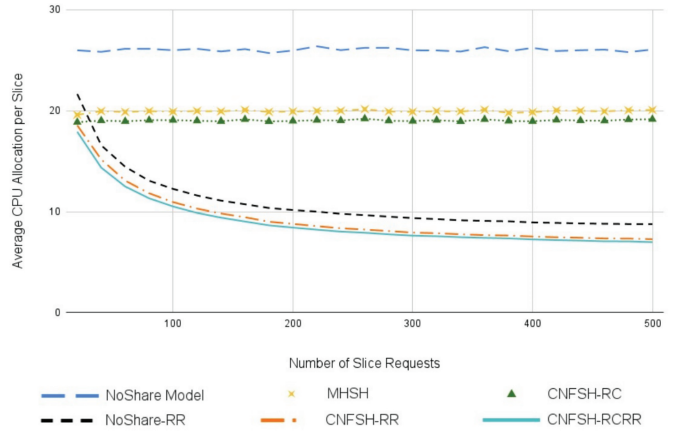


Fig. 7. Resource Usage without Delay-Aware Heuristic vs. Number of Slice Requests.

ascending and descending sorting of the remaining capacities of nodes in order to maximize SSR with delay awareness.

In Fig. 6, the y-axis shows the percentage of change in the number of SSR when the delay-aware heuristic is applied. D-CNFSH is considered the delay-aware version of MSHH. It is notable that RR schemes are less affected by delay constraints compared to CNFSH schemes without the RR heuristic, due to the fact that RR schemes prefer network slices with a low number of NFs, and these network slices are less likely to be rejected due to delay constraints.

In Fig. 7 and 8, the y-axes show the average CPU allocated to each satisfied slice. CNFSH-RC has 28% and 5% less resource consumption compared to the NoShare Model and MSHH, respectively. While CNFSH-RCRR uses 20% less resources than NoShare-RR (Fig. 7). Resource usage increases when the delay-aware heuristic is applied, as seen in Fig. 7 and 8. For instance, the CPU allocation to network slices increases by around 9% and 5% with D-CNFSH-RR and D-CNFSH-RCRR, respectively, compared to their delay-tolerant versions with 500 slice requests (Fig. 7 and 8).

In Fig. 9, our novel metric, the ratio of underutilized resources to allocated resources on the y-axis, is around 30% in the D-NoShare and D-NoShare-RC models and drops below

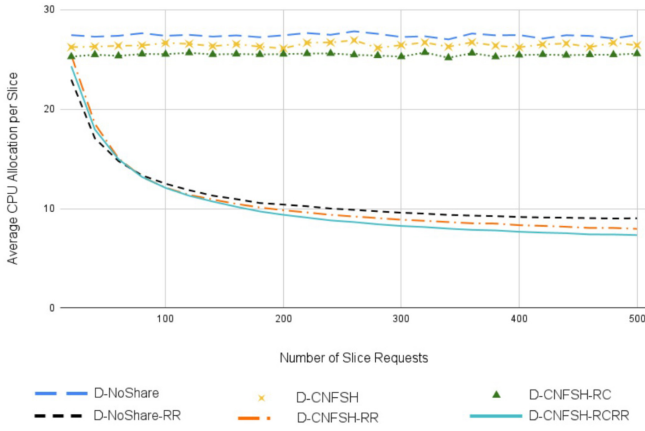


Fig. 8. Resource Usage with Delay-Aware Heuristic vs. Number of Slice Requests.

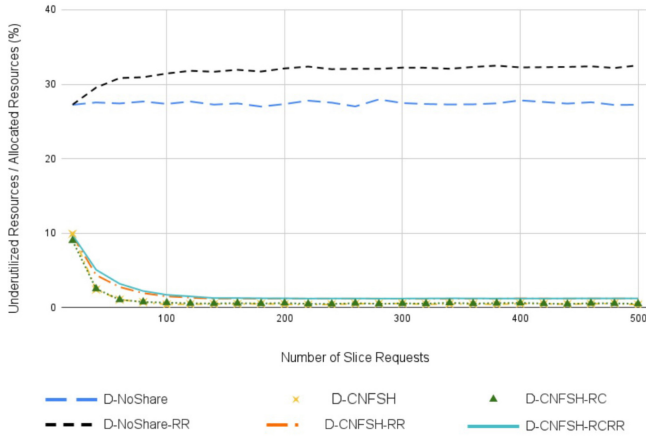


Fig. 9. The percentage of underutilized resources to allocated resources vs. number of slice requests.

1.3% in all sharing models. Moreover, this ratio is around 0.4% with D-CNFSH, around 0.5% with D-CNFSH-RC, and around 1.2% with both D-CNFSH-RR and D-CNFSH-RCRR. The high ratio with RR schemes is due to the fact that in each slice request, the NF with the heaviest workload, UPF, can take place only once. UPF causes the highest amount of underutilized resources. As a result, because of the higher number of SSR with RR, the ratio of underutilized resources to allocated resources becomes less without RR. Our proposed metric, the ratio of underutilized resources to allocated resources, investigates how much energy is wasted per energy used. In that sense, it is worth using this metric in environmentally sensitive and green applications.

The experiments with delay-aware schemes show that deploying delay-aware slices comes with the cost of extra computing resources necessary to achieve the same number of satisfied requests with delay-tolerant ones. As seen in Fig. 4, 5 and 6, the performance loss of D-CNFSH-RR and D-CNFSH-RCRR is around 9% and 5% compared to CNFSH-RR and CNFSH-RCRR, respectively, for 500 slice requests. However, in D-CNFSH and D-CNFSH-RC, the performance loss is higher than 20% for the same number of requests, compared to MSHS and CNFSH-RC, respectively. This is due to the fact that the RR schemes favor network slices with a low number of

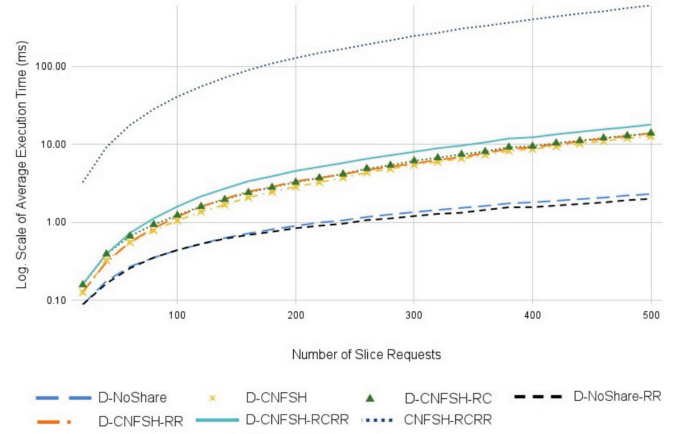


Fig. 10. The logarithmic scale of Average Execution Time (ms) vs. Number of Slice Requests.

NFs. Therefore, the delay requirements of a few NFs result in a lower number of rejected slice requests compared to schemes without RR.

Another cost of delay-awareness is the sorting of computing nodes for almost each slice. However, the complexity of $O(N \log N)$ for a small group of nodes, which is 6 in our experiments, is negligible.

Apart from the discussion on delay-aware schemes, we have not focused on the effects of the bandwidth constraint in performance evaluations. A diverse range of design parameters for bandwidth, as well as delay and CPU constraints, can be experimented with in order to see how the proposed schemes are affected in terms of their performance. We leave this issue for a future study.

Moreover, our methodology is based on a few assumptions. First, the list of slice requests is static and never changes during runtime. Experiments with online slice scheduling and additional analysis are on our roadmap. Another point of discussion is that adopting a sorting method that takes $O(S \log S)$ time may reduce the scheduler's performance as the number of slice requests increases. However, with the longest slice request lists we used, D-CNFSH-RCRR completes its execution in roughly 17.9 ms, as seen in Fig. 10, offering a more than 29% increase in slice satisfaction numbers than D-NoShare-RR, which is executed in 2 ms, corresponding to a 795% increase in execution time. Meanwhile, D-CNFSH-RC achieves an 18% higher slice satisfaction number compared to D-NoShare, with a 504% increase in execution time. Fig. 10, shows the logarithmic scale of execution times of delay-aware schemes in the y-axis.

Lastly, we have accelerated the code base of CNFSH [1], with in-memory allocations, and achieved around a 97% decrease in execution time. As seen in Fig. 10, CNFSH-RCRR completes its execution in 603 ms, while D-CNFSH-RCRR completes the same in 17.9 ms on average.

B. Performance Evaluation of the Priority-Aware LB

In this subsection, we demonstrate how our priority-aware load balancing mechanism provides performance isolation among neighboring network slices that share the same NF.

TABLE III
DELAY MEASUREMENTS FOR SCALING AND FAIL-OVER SCENARIOS WITH PRIORITY-AWARE LB

Priority	Scenarios	Failure Detection (Worst Case)	LB USR Processing (Worst Case)	Repeat NF Placement	Running NF Instance	Total Delay
High-Priority Network Slice	VM Traffic Congestion	-	0.105 ms** [29]	-	-	0.105 ms
	VM Failure	-	0.105 ms** [29]	-	-	0.105 ms
	Pod Traffic Congestion	-	0.105 ms** [29]	-	-	0.105 ms
	Pod Failure	-	0.105 ms** [29]	-	-	0.105 ms
Low-Priority Network Slice	VM Traffic Congestion	-	0.105 ms [29]	0.008 ms	11.489 s [11]	11.489 s
	VM Failure	3.199 ms*	0.105 ms [29]	-	11.489 s [11]	11.492 s
	Pod Traffic Congestion	-	0.105 ms [29]	0.008 ms	1.129 s [11]	1.129 s
	Pod Failure	3.199 ms*	0.105 ms [29]	-	1.129 s [11]	1.132 s

*Including 2 ms health check interval

**Assuming that the high-priority slice does not exceed its SLA quota

The main advantage of our proposed setting is that we use CNFs instead of VNFs, which are common in the literature. However, we also provide measurements for VNFs in case our LB is integrated into a VNF-based setting. In [11], Frangoudis et al. show that running a VM takes 11.5 seconds on average. Mesbahi et al. [40] show that even passing the role of an active VM to an already running passive VM takes 2 seconds. Using our priority-aware LB, the latency to detect a congestion (0.105 ms [29]), start a container (1.129 seconds [11]), and then direct the necessary traffic to that container (0.105 ms [29]) takes around 1.129 seconds, as shown in Table III. The LB runs on Windows 8.1 with 512 MB of RAM [29] and the container is on Ubuntu 14.10 with a 16-core Intel Xeon processor and 32 GB of RAM [11]. Apart from the low latency, our mechanism is energy-saving since there is no need to start a redundant VM or a container beforehand; new containers are triggered in case of network congestion or pod failure.

Table III shows delay calculations of our proposed LB, based on the measurements of [11], [29], and this paper, in case of congestion and NF failure scenarios for both VM and pod deployments. Table III considers the usage of priority-aware LB with both VMs and pods, so that the related measurements are based on Algorithm 2. The NF placement delay is based on the average execution time of Algorithm 1 considering D-CNSH-RCRR scheme, which has the highest delay among other aforementioned schemes. For all scenarios, the extra delay for high-priority slices is 0.105 ms due to the strict prioritization mechanism of the proposed LB.

The USRs of low-priority slices, on the other hand, are directed to the waiting queue in case of congestion. Each USR waits for a timeout period in the waiting queue, and later NF placement can be repeated in case the congestion takes longer than the network slice can tolerate. In this case, the delay can take around 11.489 seconds with a VM deployment and only 1.129 seconds with a pod deployment. In the case of a VM failure or a pod failure, the delay can be around 11.492 seconds or 1.132 seconds, respectively.

VII. CONCLUSION

Cloud-native solutions offer great potential to satisfy the requirements of 5G use cases. Availability, performance isolation, and delay requirements are the most important of these needs since they improve QoS and enable mission-critical

applications. For these purposes, redundant resources are devoted, which causes underutilized resources. In this work, we extend our study in [1], such that we introduce a delay model, a bandwidth-sharing model, and a heuristic algorithm to enable the placement of delay-sensitive network slices in polynomial time. Moreover, we introduce a performance isolation mechanism for shared NFs using our priority-aware LB. In addition, we propose a novel metric, the ratio of underutilized resources to allocated resources, which demonstrates the effect of sharing mechanisms in reducing underutilized resources. Using D-CNFSH, the ratio of underutilized resources to allocated resources reaches around 0.4%, while this ratio is around 30% for NoShare schemes. We conducted our experiments with 10 core and 2 RAN NFs. Hence, to our knowledge, D-CNFSH is the first NF sharing mechanism that targets 12 realistic 5G NFs. We have also improved the CNFSH [1] code base by using in-memory allocations, such that the execution time has decreased by 97%. Consequently, D-CNFSH is designed as a reliable and scalable solution for 5G NF sharing, relying on the LB we propose and the capabilities of cloud-native environments. We encourage the following research topics in order to further prove the applicability of D-CNFSH and improve its performance: a) A thorough investigation of the effects and scalability options of NF sharing in dynamic environments. b) Investigation of delay, bandwidth, and other parameters on the performance of NF sharing. c) The use of machine learning methods for prediction of network slice requests and network behavior. d) Investigation of the security implications of NF sharing. e) Real life applications of 5G use cases that use shared NFs.

ACKNOWLEDGMENT

The author Ömer Zekvan Yılmaz would like to thank TÜBİTAK BİDEB 2211/A for their valuable support.

REFERENCES

- [1] Ö. Z. Yılmaz and F. Alagöz, "Shared network function placement for cloud native 5G core networks," in *Proc. 14th Int. Conf. Netw. Future (NoF)*, 2023, pp. 103–107.
- [2] A. Mohamad and H. S. Hassanein, "On demonstrating the gain of SFC placement with VNF sharing at the edge," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2019, pp. 1–6.
- [3] D. Basu, S. Kal, U. Ghosh, and R. Datta, "SoftDrone: Softwarized 5G assisted drone networks for dynamic resource sharing using machine learning techniques," *Comput. Electr. Eng.*, vol. 101, Jul. 2022, Art. no. 107962.

- [4] B. Yi, X. Wang, M. Huang, S. K. Das, and K. Li, "Fairness-aware VNF sharing and rate coordination for high efficient service scheduling," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4597–4611, Dec. 2022.
- [5] H. Chergui and C. Verikoukis, "Offline SLA-constrained deep learning for 5G networks reliable and dynamic end-to-end slicing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 350–360, Feb. 2020.
- [6] A. J. Zharabad, S. Yousefi, and T. Kunz, "Network slicing in virtualized 5G core with VNF sharing," *J. Netw. Comput. Appl.*, vol. 215, Jun. 2023, Art. no. 103631.
- [7] M. Liyanage et al., "A survey on zero touch network and service management (ZSM) for 5G and beyond networks," *J. Netw. Comput. Appl.*, vol. 203, Jul. 2022, Art. no. 103362.
- [8] D. M. Gutierrez-Estevez et al., "Artificial intelligence for elastic management and orchestration of 5G networks," *IEEE Wireless Commun.*, vol. 26, no. 5, pp. 134–141, Oct. 2019.
- [9] A. Gupta and R. K. Jha, "A survey of 5G network: Architecture and emerging technologies," *IEEE Access*, vol. 3, pp. 1206–1232, 2015.
- [10] O. Arouk and N. Nikaein, "Kube5G: A cloud-native 5G service platform," in *Proc. IEEE Glob. Commun. Conf.*, 2020, pp. 1–6.
- [11] P. A. Frangoudis, L. Yala, and A. Ksentini, "CDN-as-a-service provision over a telecom operator's cloud," *IEEE Trans. Netw. Service Manag.*, vol. 14, no. 3, pp. 702–716, Sep. 2017.
- [12] C. De Alwis, P. Porambage, K. Dev, T. R. Gadekallu, and M. Liyanage, "A survey on network slicing security: Attacks, challenges, solutions and research directions," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 1, pp. 534–570, 1st Quart., 2024.
- [13] R. F. Olimid and G. Nencioni, "5G network slicing: A security overview," *IEEE Access*, vol. 8, pp. 99999–100009, 2020.
- [14] *Telecommunication Management; Network Sharing; Concepts and Requirements*, 3GPP Standard TS 32.130, 2016.
- [15] N. Van Giang and Y. H. Kim, "Slicing the next mobile packet core network," in *Proc. 11th Int. Symp. Wireless Commun. Syst. (ISWCS)*, 2014, pp. 901–904.
- [16] K. Kalkan, L. Altay, G. Gür, and F. Alagoz, "JESS: Joint entropy-based DDoS defense scheme in SDN," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2358–2372, Oct. 2018.
- [17] B. Yigit, G. Gur, B. Tellenbach, and F. Alagoz, "Secured communication channels in software-defined networks," *IEEE Commun. Mag.*, vol. 57, no. 10, pp. 63–69, Oct. 2019.
- [18] D. Dietrich, C. Papagianni, P. Papadimitriou, and J. S. Baras, "Network function placement on virtualized cellular cores," in *Proc. 9th Int. Conf. Commun. Syst. Netw. (COMSNETS)*, 2017, pp. 259–266.
- [19] X. Chen, L. Rupperecht, R. Osman, P. Pietzuch, F. Franciosi, and W. Knottenbelt, "CloudScope: Diagnosing and managing performance interference in multi-tenant clouds," in *Proc. IEEE 23rd Int. Symp. Model., Anal., Simul. Comput. Telecommun. Syst.*, 2015, pp. 164–173.
- [20] O. M. Bushnaq, I. Zhilin, G. De Masi, E. Natalizio, and I. Akyildiz, "Automatic network slicing for resource allocation in underwater acoustic communication systems," in *Proc. Int. Balkan Conf. Commun. Netw. (BalkanCom)*, 2021, pp. 90–95.
- [21] S. Arora and A. Ksentini, "Dynamic resource allocation and placement of cloud native network services," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.
- [22] T. Truong-Huu, P. M. Mohan, and M. Gurusamy, "Service chain embedding for diversified 5G slices with virtual network function sharing," *IEEE Commun. Lett.*, vol. 23, no. 5, pp. 826–829, May 2019.
- [23] S. Vittal and A. A. Franklin, "HARNESS: High availability supportive self reliant network slicing in 5G networks," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 1951–1964, Sep. 2022.
- [24] E. A. Brewer, "Kubernetes and the path to cloud native," in *Proc. 6th ACM Symp. Cloud Comput.*, 2015, pp. 167–167.
- [25] C. Carrión, "Kubernetes scheduling: Taxonomy, ongoing issues and challenges," *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1–37, 2022.
- [26] L. A. Vayghan, M. A. Saied, M. Toeroe, and F. Khendek, "Microservice based architecture: Towards high-availability for stateful applications with kubernetes," in *Proc. IEEE 19th Int. Conf. Softw. Qual., Rel. Secur. (QRS)*, 2019, pp. 176–185.
- [27] N. Poulton, *The Kubernetes Book*. Macclesfield, U.K.: Nigel Poulton Ltd., 2023.
- [28] *IMT Vision Framework and Overall Objectives Of The Future Development of IMT for 2020 and Beyond*, ITU-Rec. M.2083-0, Int. Telecommun. Union, Geneva, Switzerland, 2015.
- [29] S. Ghosh and C. Banerjee, "Priority based modified throttled algorithm in cloud computing," in *Proc. Int. Conf. Invent. Comput. Technol. (ICICT)*, 2016, pp. 1–6.
- [30] S. Yang, F. Li, S. Trajanovski, X. Chen, Y. Wang, and X. Fu, "Delay-aware virtual network function placement and routing in edge clouds," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 445–459, Feb. 2021.
- [31] V. Cacchiani, M. Iori, A. Locatelli, and S. Martello, "Knapsack problems—An overview of recent advances. Part II: Multiple, multidimensional, and quadratic knapsack problems," *Comput. Oper. Res.*, vol. 143, Jul. 2022, Art. no. 105693.
- [32] E. Amaldi, S. Coniglio, A. M. Koster, and M. Tieves, "On the computational complexity of the virtual network embedding problem," *Electron. Notes Discr. Math.*, vol. 52, pp. 213–220, Jun. 2016.
- [33] Y. Vimont, S. Boussier, and M. Vasquez, "Reduced costs propagation in an efficient implicit enumeration for the 01 multidimensional knapsack problem," *J. Comb. Optim.*, vol. 15, no. 2, pp. 165–178, 2008.
- [34] R. Panigrahy, K. Talwar, L. Uyeda, and U. Wieder, "Heuristics for vector bin packing," microsoft.com. 2011. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/heuristics-for-vector-bin-packing/>
- [35] J. S. Ang, C. Cao, and H.-Q. Ye, "Model and algorithms for multi-period sea cargo mix problem," *Eur. J. Oper. Res.*, vol. 180, no. 3, pp. 1381–1393, 2007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377221706003675>
- [36] Ö. Z. Yılmaz, "Slicescheduler." [Online]. Available: <https://github.com/omerzekvan/SliceScheduler>
- [37] "Study on new radio access technology: Radio access architecture and interfaces, Version 14," 3GPP, Sophia Antipolis, France, Rep. TR 38.801, Mar. 2017.
- [38] H. T. Nguyen, T. Van Do, and C. Rotter, "Scaling UPF instances in 5G/6G core with deep reinforcement learning," *IEEE Access*, vol. 9, pp. 165892–165906, 2021.
- [39] *5G Security Recommendations Package# 2: Network Slicing*, NGMN, Düsseldorf, Germany, 2016.
- [40] M. R. Mesbahi, M. Hashemi, and A. M. Rahmani, "Performance evaluation and analysis of load balancing algorithms in cloud computing environments," in *Proc. 2nd Int. Conf. Web Res. (ICWR)*, 2016, pp. 145–151.



Ömer Zekvan Yılmaz received the B.Sc. and M.Sc. degrees in computer engineering from Sabanci University in 2006 and 2009, respectively. He is currently pursuing the Ph.D. degree with Boğaziçi University. He is a Lead Software Engineer with ULAK Communications. His research interests include network orchestration, network security, next-generation communication networks, and resource allocation.



Fatih Alagoz received the B.Sc. degree in electrical engineering from Middle East Technical University, Türkiye, in 1992, and the D.Sc. degree in electrical engineering from The George Washington University, USA, in 2000. He is a Professor with the Computer Engineering Department, Boğaziçi University, Türkiye. His current research areas include network security, next generation wireless/mobile/satellite networks, and AI applications across six industries, including finance, telecom, CRM, tourism, health care, and software development.